

Devoir

Moonlighter

Léo Simon

1.

Le jeu est un dungeon crawler, rpg, rogue like en pixel art et a la vue isométrique. Le joueur doit parcourir des donjons pour récupérer des objets et des trésors qu'il va devoir ensuite vendre dans son magasin en journée.

2.

Les mécaniques de gameplay que j'ai rencontré au bout d'une heure de jeu.

Déplacement	déplacement sans possibilité de sprint.
Roulade	Offre des frames d'invincibilité et permet de passer les trous
Génération procédurale	Les donjon sont générés procéduralement à chaque entrée.
Ennemis	Des ennemis se trouvent dans les donjons, ils peuvent être tués pour récupérer des ressources.
Inventaire	Permet de stocker des objets pour les ramener ensuite hors des donjons.
Attaquer et parier	deux armes peuvent être équipées en même temps et il est possible des switcher entre les deux. Si un bouclier est équipé, alors il est possible de parier les coups des monstres sinon il est possible de porter une attaque chargée.
Médaille	Permet contre 300 d'or de quitter le donjon en gardant le butin amassé.
Mise en vente et gestion des prix des objets	En fonction de l'offre, la demande et la rareté des objets le prix doit être ajusté.
Feed back sur la justesse des prix	les clients réagissent à la justesse des prix ce qui permet de les ajuster.
Amélioration de la ville et du magasin	permet de débloquent d'autre magasins ou d'améliorer le siens contre de l'argent

3.

Le jeu se déroule en deux phase :

La première phase est l'exploration de donjon sous la forme d'un dungeon crawler avec des éléments de rogue like puisqu'il n'est pas attendu du joueur de finir le donjon du premier coup mais plutôt de le quitter en avance avec son butin pour le vendre, s'améliorer et revenir plus fort.

La deuxième phases est celle du magasin et de la ville :

Dans cette phase le joueur doit gérer son magasin, l'ouvrir à la clientèle, mettre ses articles en vente et gérer les prix en fonction de l'offre et de la demande.

Mais il peut aussi se déplacer dans la ville pour débloquer des magasins qui vont lui permettre de s'améliorer.

4

Le type de génération aléatoire qu'utilise Moonlighter est de la génération procédurale :

Le jeu dispose de salles de donjons déjà créées ainsi que des monstres. Lorsque le joueur rentre dans un donjon le jeu assemble des pièces de manière aléatoire mais en faisant en sorte que :

- toutes les portes d'une salle mènent à une autre
- qu'il n'y ai pas deux fois la même salle
- La salle de bosse ne doit pas se trouver à côté de la porte d'entrée
- Un salle secrète doit apparaître dans l'un des trous dans l'une des salle
- Une salle de fontaine de soins doit apparaître et sera toujours la salle précédant la salle du boss
- La salle du campement abandonné doit toujours être présente dans le donjon

5.

certain objets que l'on récupère dans les donjons possèdent des malédictions qui on un effet dans l'inventaire :

- ne pouvoir se placer qu'au fond ou sur l'un des bord de l'inventaire
- ne pas pouvoir être identifié avant la sortie du donjon
- détruire un objets situé sur une case à côté dans une direction aléatoire
- s'autodétruire si le joueur prend trop de dégâts

D'autres sont des bonus :

- Retirer un malus situé sur une case à côté dans une direction aléatoire
- Copier un objets situé sur une case à côté dans une direction aléatoire, à la sortie du donjon
- envoyer un objet situé sur une case à côté dans une direction aléatoire directement dans le coffre du magasin.

En terme de prog :

L'inventaire est un tableau composé de 20 indexs (a moins d'être amélioré). Lorsqu'un objets est déposé le jeux vérifie un index du tableau en fonction de l'orientation de la flèche pour faire l'effet du buff/débuff en se basant sur l'index sur lequel on a déposé l'objet :

- Si la flèche indique à droite alors la vérification se fait sur : l'index sur lequel on a déposé l'objet + 1
- Si la flèche indique à gauche alors la vérification se fait sur : l'index sur lequel on a déposé l'objet - 1
- Si la flèche indique en diagonale haut droite alors la vérification se fait sur : l'index sur lequel on a déposé l'objet - 9
- Si la flèche indique en diagonale haut gauche alors la vérification se fait sur : l'index sur lequel on a déposé l'objet - 11
- Si la flèche indique en diagonale bas gauche alors la vérification se fait sur : l'index sur lequel on a déposé l'objet + 9
- Si la flèche indique en diagonale bas droite alors la vérification se fait sur : l'index sur lequel on a déposé l'objet + 11
- Si la flèche indique en haut alors la vérification se fait sur : l'index sur lequel on a déposé l'objet - 10
- Si la flèche indique en bas alors la vérification se fait sur : l'index sur lequel on a déposé l'objet + 10

6.

Lors de la mise en vente d'un objet, le joueur doit choisir la quantité qu'il place et le prix. Lors de la première mise en vente d'un objet il faut essayer de deviner le prix adéquat en fonction de la position de l'objet dans le carnet du joueur. Lors de la sélection du prix le joueur voit le montant total

Les clients vont ensuite déambuler dans le magasin et réagir au prix qui a été fixé ce qui donne une indication au joueur si le prix était trop haut ou trop bas qu'il va pouvoir ensuite adapter.

Un objet a aussi une popularité qui indique à quel point l'engouement et donc la demande pour cet objet est élevé : Plus il est haut, plus le prix de l'objet peut être augmenté.

7.

J'ai trouvé le concept des deux phases de gameplay très intéressant avec une bonne synergie. Le jeu m'a rapidement pris et j'ai eu envie de progresser. J'ai presque trouvé la phase de vente plus amusante que la phase d'exploration. Je pense que le jeu pourrait rapidement se lasser mais il est possible que j'essaye de le finir si j'en ai le temps.

Partie Pratique :

1.

J'ai choisi de faire une mécanique pour ramasser des objets au sol et les stocker dans un inventaire. Je pense qu'il est pertinent de commencer à apprendre à travailler sur des inventaire vu qu'il s'agit d'une mécanique omniprésente dans les jeux vidéos et sous des formes extrêmement diverses.

2.

Dans un premiers temps je crée deux widget blueprint :

Le premier "WB_Inventory" sert de visuel pour l'inventaire et le second "WB_Item_Slot" représentent les différents emplacements qui composeront l'inventaire.

Dans "WB_Item_Slot" je m'assure juste que l'image représentée soit la bonne et que la qté d'un item dans l'inventaire n'est indiqué que si elle est supérieur à 1.

Dans "WB_Inventory" je génère avec une boucle for les 20 "WB_Item_Slot" affiché dans l'inventaire en les ajoutant dans la Wrap box du Widget inventaire. cette fonction va aussi me permettre d'ajouter des item grâce à l'input Inventaire (variable "BP_Component_Inventory") de la fonction qui va me permettre de récupérer le tableau qui va stocker mes items pour mettre a jours correctement l'inventaire

Ensuite je crée un BP "BP_Item", les objets que l'on va pouvoir ajouter à notre inventaire et une structure "S_Inventory" qui contient toutes les informations de nos objets a savoir :

- leur nom
- si ils peuvent être stacks
- la quantité
- l'image qu'il faudra afficher dans l'inventaire
- leur mesh

Dans le BP objet je lui ajoute une variable "S_inventory" qui est instance editable et exposed on spawn afin de pouvoir personnaliser chaque objets. Enfin en construction script je set le Static Mesh de l'item pour être celui contenu dans sa variable "S_inventory".

Pour faire l'inventaire je crée un Actor component "BP_Component_Inventory" que j'ajoute à mon player.

Il possède une variable "S_inventory" nommé "item" en array qui va permettre de contenir toutes les infos de tous les items.

En begin play je génère le widget "WB_Inventory", set la taille de l'array "item" à 20 (les 20 cases de l'inventaire) et je lance la fonction du widget inventaire pour générer les 20 cases dans lesquels pourront aller les objets.

J'ai ensuite créé une fonction avec l'input mapping context pour faire en sorte que quand on appuie sur Tab alors l'inventaire s'ouvre et quand on appuie dessus l'inventaire se ferme.

Une deuxième fonction d'imput (set sur E) permet elle de ramasser les objets : grâce à un sphère trace by channel qui s'applique à nos pied on effectue un cast pour to BP_item, si le cast est réussi on fait une série de vérification grâce à une boucle for each sur l'array "item" :

- l'objet est-il déjà présent dans l'inventaire ? (comparaison de nom)
- l'objet peut-il être stack ?
- la quantité de l'objet est <5 (limite de cumul d'un même objet dans moonlighter)

Si toutes ces vérification sont valides alors on met à jour la nouvelle quantité de l'objet.

Si l'objet était déjà présent et non stackable ou déjà à 5 dans une case alors la boucle vérifie si la qté d'un objet présent dans une case est = 0 car cela signifie que la case est vide et que donc on peut ajouter l'objet dans cette case.

3.

Selon moi récupérer des objets pour les ajouter dans son inventaire est l'une des mécaniques primordiales du jeu. Cela permet de transporter les objets au magasin pour ensuite les vendre. Cette mécanique de remplir son inventaire d'objets que l'on va ensuite vendre ou utiliser est le pilier de Moonlighter. Elle est aussi agrémentée d'autres mécaniques comme les malus ou bonus sur les objets que l'on ramasse qui vont influencer notre manière de les disposer dans notre inventaire.

4.

J'ai aimé la manière qu'avait Moonlighter d'utiliser quelque chose d'aussi courant que l'inventaire pour en faire quelque chose de plus unique et inhabituel. J'ai aussi estimé que ce devoir était une bonne occasion pour apprendre à faire et utiliser des inventaires vu qu'il s'agit d'une mécanique qui risque très probablement de revenir dans les projets sur lesquels je vais travailler à l'avenir.